INTRODUCTION TO PYTHON
UNIT 6: REAL-WORLD ENGINEERING APPLICATIONS & DSP SIMULATION
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja & Press et al.

>> 6.1 Numerical Methods in Electrical Engineering

Many real-world engineering models (e.g. non-linear diode equations, transient differential equations) lack analytical closed-form solutions and mandate computational algorithms:
* 1. Non-Linear Root Finding (Newton-Raphson Method):
- Iterative formulation: x_{n+1} = x_n - f(x_n) / f'(x_n). Quadratic rate of convergence near roots.
* 2. Numerical Integration:
- Trapezoidal Rule: Integral_a^b [f(x) dx] approx (h / 2) * [f(a) + 2 * sum(f(x_i)) + f(b)].
- Simpson's 1/3 Rule: Fits parabolic quadratic polynomials through sets of 3 points: Integral_a^b [f(x) dx] approx (h / 3) * [f(x_0) + 4 * sum(odd) + 2 * sum(even) + f(x_n)]. Higher accuracy order O(h^4).

>> 6.2 Filter Simulation and Frequency Response Modeling

* Continuous-Time Transfer Function: H(s) = V_out(s) / V_in(s).
* First-Order Low-Pass RC Filter: H(j*omega) = 1 / (1 + j * omega * R * C). Magnitude |H(j*omega)| = 1 / sqrt(1 + (omega / omega_c)^2).
* Bode Plot Representation: Decibel magnitude response: Gain (dB) = 20 * log10(|H(j*omega)|). Cutoff frequency f_c = 1 / (2 * pi * R * C) corresponds to -3.01 dB attenuation.

>> 6.3 Spectral Analysis via Discrete Fourier Transform (DFT / FFT)

* Time-Domain to Frequency-Domain Transformation: The Fast Fourier Transform (FFT) computes the N-point Discrete Fourier Transform in O(N log N) operations compared to O(N^2) for brute-force DFT.
* Discrete Fourier Transform Definition: X[k] = sum_{n=0}^{N-1} x[n] * e^(-j * 2 * pi * k * n / N).
* Nyquist Frequency: Maximum resolvable frequency = f_s / 2. Frequencies above Nyquist alias down unless filtered.

>> 6.4 Communication Packet Framing and CRC-16 Checksum Verification

* Data Link Reliability: Ensuring uncorrupted packet transmission across noisy serial channels.
* Cyclic Redundancy Check (CRC-16): Polynomial binary division treating bitstreams as algebraic polynomials. Detects all single-bit errors, all double-bit errors, and burst errors of length <= 16 bits.

[PROGRAM 1: Non-Linear Diode Circuit Operating Point via Newton-Raphson]

Problem Context: Solve for the exact quiescent diode voltage V_D in a series circuit with battery V_S = 5.0 V, resistor R = 1.0 kOhm, and a Silicon diode (Shockley parameters: I_S = 1e-14 A, thermal voltage V_T = 0.026 V).
Python Source Code:

```
import math

def solve_diode_operating_point(v_s, r, i_s=1e-14, v_t=0.026, tol=1e-7, max_iter=100):
# Circuit KVL: f(V_D) = (V_S - V_D) / R - I_S * (exp(V_D / V_T) - 1) = 0
# Derivative: f'(V_D) = -1/R - (I_S / V_T) * exp(V_D / V_T)

v_d = 0.7 # Initial engineering guess for silicon diode
for i in range(max_iter):
f_val = (v_s - v_d) / r - i_s * (math.exp(v_d / v_t) - 1)
f_prime = -1.0 / r - (i_s / v_t) * math.exp(v_d / v_t)

delta = f_val / f_prime
v_d_new = v_d - delta

if abs(v_d_new - v_d) < tol:
current_ma = ((v_s - v_d_new) / r) * 1000.0
return v_d_new, current_ma, i + 1
v_d = v_d_new
raise RuntimeError('Newton-Raphson failed to converge')

vd, id_ma, iterations = solve_diode_operating_point(v_s=5.0, r=1000.0)
print(f'Newton-Raphson Converged in {iterations} iterations:')
print(f' Diode Forward Voltage (V_D) : {vd:.6f} Volts')
print(f' Circuit Current (I_D) : {id_ma:.6f} mA')
```

[PROGRAM 2: Simpson's 1/3 Rule for Energy Dissipation Integration]

Problem Context: Calculate the total energy dissipated in Joules by an electrical load carrying a time-varying current i(t) = 10 * sin(100 * pi * t) through a 5 Ohm resistor

over half a cycle (0 to 10 ms).
Python Source Code:

```
import math

def simpson_energy(r, duration, n_intervals=100):
if n_intervals % 2 != 0:
n_intervals += 1 # Simpson’s rule mandates even number of intervals

h = duration / n_intervals

# Power function: p(t) = i(t)^2 * R
def p(t):
i = 10.0 * math.sin(100.0 * math.pi * t)
return (i**2) * r

# Simpson’s 1/3 integration summation
sum_val = p(0) + p(duration)
for i in range(1, n_intervals):
t_i = i * h
weight = 4 if (i % 2 != 0) else 2
sum_val += weight * p(t_i)

energy_joules = (h / 3.0) * sum_val
return energy_joules

duration_s = 0.010 # 10 milliseconds
e_numeric = simpson_energy(r=5.0, duration=duration_s, n_intervals=200)
# Analytical check: Integral_0^0.01 [ 500 * sin^2(100 pi t) dt ] = 500 * (0.01 / 2) =
2.5 Joules
print(f’Total Energy Dissipated (Simpson 1/3) : {e_numeric:.6f} Joules’)
print(f’Exact Analytical Integration Value : 2.500000 Joules’)
```

[PROGRAM 3: Fast Fourier Transform (FFT) Spectral Analysis Simulator]

Problem Context: Synthesize a dual-tone audio signal (500 Hz + 1200 Hz) corrupted by random Gaussian noise, and extract dominant harmonic peaks using NumPy FFT.
Python Source Code:

```
import numpy as np

# Sampling parameters
f_s = 8000.0 # 8 kHz audio sampling rate
n_samples = 2048 # FFT length (power of 2)
t = np.arange(n_samples) / f_s
```

```
# Composite dual-tone signal with added white noise
signal_clean = 2.0 * np.sin(2 * np.pi * 500 * t) + 1.2 * np.sin(2 * np.pi * 1200 * t)
np.random.seed(42)
noise = np.random.normal(0, 0.8, size=n_samples)
signal_noisy = signal_clean + noise

# Compute Fast Fourier Transform (FFT)
fft_complex = np.fft.fft(signal_noisy)
fft_magnitude = (2.0 / n_samples) * np.abs(fft_complex[:n_samples // 2])
frequencies = np.fft.fftfreq(n_samples, 1.0 / f_s)[:n_samples // 2]

# Identify top 2 frequency peaks
peak_indices = np.argsort(fft_magnitude)[-2:][::-1]
print('FFT Dominant Frequency Detection:')
for idx in peak_indices:
print(f' Detected Peak: {frequencies[idx]:7.1f} Hz (Amplitude:
{fft_magnitude[idx]:.2f})')
```

[PROGRAM 4: End-to-End CRC-16 Communication Packet Verifier]

Problem Context: Implement a complete serial transmitter and receiver module that computes CRC-16 (CCITT standard: polynomial 0x1021) to detect transmission errors.

Python Source Code:

```
def compute_crc16_ccitt(data_bytes):
crc = 0xFFFF
for byte in data_bytes:
crc ^= (byte << 8)
for _ in range(8):
if crc & 0x8000:
crc = ((crc << 1) ^ 0x1021) & 0xFFFF
else:
crc = (crc << 1) & 0xFFFF
return crc

# 1. Transmitter Side: Prepare packet and append CRC-16
payload = b"CMD_MOTOR_ON_SPEED_1500"
crc_tx = compute_crc16_ccitt(payload)
transmitted_packet = payload + crc_tx.to_bytes(2, byteorder='big')

# 2. Receiver Side (Uncorrupted Packet):
rx_payload = transmitted_packet[:-2]
rx_crc_claimed = int.from_bytes(transmitted_packet[-2:], byteorder='big')
rx_crc_calc = compute_crc16_ccitt(rx_payload)
is_clean = (rx_crc_claimed == rx_crc_calc)
```

```
print(f'Clean Transmission Test : PASSED={is_clean} (CRC: {rx_crc_calc:#06x})')

# 3. Receiver Side with Simulated Channel Noise Bit Flip:
corrupted_packet = bytearray(transmitted_packet)
corrupted_packet[5] ^= 0x01 # Flip 1 bit in payload
rx_corrupt_payload = corrupted_packet[:-2]
rx_corrupt_crc = int.from_bytes(corrupted_packet[-2:], byteorder='big')
is_corrupt_clean = (rx_corrupt_crc == compute_crc16_ccitt(rx_corrupt_payload))
print(f'Corrupted Bit Test : REJECTED={not is_corrupt_clean} (Error detected cleanly!)')
```